Group 96M. First Partial exam. March 12, 2026.

Name,Surname: ……

Small group number: …….

Instructions:

- Write your name and your small group number on this exam sheet.
- The duration of the exam is **80 minutes**.
- Mobiles must be switched off and can never be on the table.
- Use draft papers to prepare your solution. When you are satisfied, copy your final solution on this exam sheet.
- Take great care in the presentation of your solution. It is essential that your code is clear, well-organized, and well-structured.
- You are not allowed to ask your colleagues for a pen, pencil, etc. If you need something, ask the teacher for it.

Exercise #1: (1.0 point)

Based on the implementations of the Stack class, implement a **replace_stack (stack, old, new)** method that takes a **stack of integers and two integer values (old, new)** as input parameters and replaces all occurrences of the **'old'** value in the stack with the **'new'** value.

Example:

Input stack = [5,1,3,2,6,5,5], **old** = 5, **new** =0

Output stack = [**0**,1,3,2,6,**0**,**0**]

What is the worst-case time complexity (Big-O) of this method? Justify your answer.

Rules:

- Your implementation must be correct, robust, and **as efficient as possible in terms of time complexity**.
- In this method, you cannot use Python lists. The solution must only use the provided **Stack** class. The solution can use auxiliary variables of the class Stack.

```python
class Stack:

    def __init__(self):
        self.items = []

    def __len__(self):
        return len(self.items)

    def isEmpty(self):
        return len(self) == 0

    def push(self, e):
        self.items.append(e)

    def pop(self):
        if self.isEmpty():
            print('Error: Stack is empty')
            return None
        return self.items.pop()

    def top(self):
        if self.isEmpty():
            print('Error: Stack is empty')
            return None
        return self.items[-1]


def ReplaceStack(s,old,new):
    ...
```

Exercise #2: (1.5 points)

Given the classes DNode and DList, implementations of node and doubly linked list, respectively, as seen in class, implement a method in DList called **remove_neighbours** that receives one value **x** as input parameter. Its function will be to locate the first occurrence of that value in the list and remove the adjacent elements, but not the one given as a parameter. If the node containing the parameter is at the start or end of the list, only the single adjacent node will be removed. If that value is not found in the list, the method should not produce any changes to it.

What is the worst-case time complexity (Big-O) of this method? Justify your answer.

Rules:

- Your implementation must be correct, robust, and **as efficient as possible in terms of time complexity**.
- If you need to use any method of the class Dist, you must provide its implementation
- Using auxiliary data structures (e.g., Python lists, dictionaries, or other linked lists) is **not allowed**.

Examples:

empty list	x=3	empty list
1	x=3	1
1	x=1	1
1<->**2**<->3<->**4**<->5<->6	x=3	1<->3<->5<->6
1<->**2**<->3<->**3**<->5<->3	x=3	1<->3<->5<->3
1<->1<->**1**<->4<->**1**<->6	x=4	1<->1<->4<->6
1<->**2**<->3<->4<->5<->6	x=1	1<->3<->4<->5<->6
1<->2<->3<->4<->**5**<->6	x=6	1<->2<->3<->4<->6

Below is the header of the method you must implement.

```python
class DNode:
  def __init__(self, elem, next=None, prev=None ):
    self.elem = elem
    self.next = next
    self.prev = prev

class DList:
  def __init__(self):
    self._head=None
    self._tail=None
    self._size=0

  def removeNeighbours(self, x):
```